

12TH AUG 2026

OOP Notes: Abstraction, Encapsulation & Arrays (Java)

1. Object-Oriented Programming — Quick Recap

OOP is a programming paradigm based on the concept of **objects**, which bundle **data (fields)** and **behavior (methods)** together.

The four pillars of OOP:

Pillar	Meaning
Encapsulation	Bundling data + methods, restricting direct access
Abstraction	Hiding implementation, exposing only essential features
Inheritance	Acquiring properties of another class
Polymorphism	One interface, many implementations

This note focuses on **Encapsulation**, **Abstraction**, and **Arrays**.

2. Encapsulation

2.1 Definition

Encapsulation is the technique of **wrapping data (variables) and code (methods) together as a single unit**, and **restricting direct access** to some of an object's components using access modifiers.

- Achieved in Java using: `private` fields + `public` getters/setters
- Also called **data hiding**

2.2 Why use it?

- Protects internal state from unintended/invalid modification
- Allows validation logic before setting a value

- Increases maintainability — internal implementation can change without breaking external code
- Improves security by controlling exactly what is exposed

2.3 Access Modifiers (relevant to encapsulation)

Modifier	Class	Package	Subclass	World
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

2.4 Code Example — Encapsulation

```
public class BankAccount {
    // private fields - hidden from outside access
    private String accountHolder;
    private double balance;

    // constructor
    public BankAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = (balance >= 0) ? balance : 0;
    }

    // getter
    public double getBalance() {
        return balance;
    }

    // getter
    public String getAccountHolder() {
        return accountHolder;
    }

    // setter with validation logic (this is the key benefit of encapsulation)
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Deposit amount must be positive.");
        }
    }
}
```

```

    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Invalid or insufficient funds.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("James", 1000.0);

        acc.deposit(500);
        acc.withdraw(200);

        // acc.balance = -5000; // NOT ALLOWED - balance is private
        System.out.println(acc.getAccountHolder() + "'s balance: " +
acc.getBalance());
    }
}

```

Key point: `balance` cannot be accessed or set directly from `Main`. All changes must go through validated methods (`deposit` , `withdraw`). This is encapsulation in action.

3. Abstraction

3.1 Definition

Abstraction means **hiding internal implementation details and showing only the essential/relevant features** of an object to the user.

| Encapsulation hides **data**. Abstraction hides **implementation complexity**.

3.2 How Java achieves Abstraction

1. **Abstract classes** (partial abstraction, 0–100%)
2. **Interfaces** (full abstraction, 100% — until Java 8 introduced default methods)

3.3 Abstract Classes

- Declared using the `abstract` keyword
- Cannot be instantiated directly
- Can have **abstract methods** (no body) and **concrete methods** (with body)
- A subclass must implement all abstract methods (unless it is also abstract)

```
abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }

    // abstract method - no implementation, subclasses MUST implement it
    abstract double area();

    // concrete method - shared by all subclasses
    void displayColor() {
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, width;

    Rectangle(String color, double length, double width) {
        super(color);
        this.length = length;
        this.width = width;
    }

    @Override
```

```

    double area() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle("Red", 5);
        Shape s2 = new Rectangle("Blue", 4, 6);

        // Shape s = new Shape("Green"); // NOT ALLOWED - abstract class

        s1.displayColor();
        System.out.println("Circle Area: " + s1.area());

        s2.displayColor();
        System.out.println("Rectangle Area: " + s2.area());
    }
}

```

3.4 Interfaces

- Declared using `interface` keyword
- All methods are `public abstract` by default (before Java 8)
- All fields are `public static final` by default
- A class uses `implements` to use an interface
- Supports **multiple inheritance** of type (a class can implement multiple interfaces)

```

interface Payable {
    void pay(double amount); // implicitly public abstract
}

interface Refundable {
    void refund(double amount);
}

// multiple interface implementation
class Transaction implements Payable, Refundable {
    @Override
    public void pay(double amount) {
        System.out.println("Paid: " + amount);
    }

    @Override

```

```

    public void refund(double amount) {
        System.out.println("Refunded: " + amount);
    }
}

public class Main {
    public static void main(String[] args) {
        Transaction t = new Transaction();
        t.pay(1500.0);
        t.refund(300.0);
    }
}

```

3.5 Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract (default/static allowed from Java 8)
Fields	Any type	public static final only
Constructor	Yes	No
Multiple inheritance	Not supported (single class extends)	Supported (implements multiple)
Access modifiers on methods	Any	public only
Use when	Classes share common code + state	Unrelated classes need a common contract

4. Arrays in Java

4.1 Definition

An array is a **fixed-size, ordered collection of elements of the same data type**, stored in **contiguous memory locations**, accessed via an index (starting at 0).

4.2 Declaration & Initialization

```

// Declaration
int[] numbers;           // preferred style
int numbers2[];         // also valid (C-style)

```

```
// Instantiation
numbers = new int[5]; // creates array of size 5, default values = 0

// Declaration + Instantiation
int[] scores = new int[5];

// Declaration + Initialization (values known upfront)
int[] marks = {90, 85, 76, 88, 92};

// Explicit form
int[] marks2 = new int[]{90, 85, 76, 88, 92};
```

4.3 Accessing & Modifying Elements

```
public class ArrayBasics {
    public static void main(String[] args) {
        int[] marks = {90, 85, 76, 88, 92};

        // access by index
        System.out.println("First element: " + marks[0]);
        System.out.println("Last element: " + marks[marks.length - 1]);

        // modify
        marks[2] = 100;

        // iterate with normal for loop
        for (int i = 0; i < marks.length; i++) {
            System.out.println("marks[" + i + "] = " + marks[i]);
        }

        // iterate with for-each loop
        for (int m : marks) {
            System.out.print(m + " ");
        }
    }
}
```

4.4 Default Values

Type	Default value
int, short, byte, long	0
float, double	0.0

Type	Default value
boolean	false
char	'\u0000'
Object references (String, etc.)	null

4.5 Multi-dimensional Arrays

```
public class MultiDimArray {
    public static void main(String[] args) {
        // 2D array (matrix) - 3 rows, 3 cols
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }

        // Jagged array (rows of different lengths)
        int[][] jagged = new int[3][];
        jagged[0] = new int[]{1};
        jagged[1] = new int[]{1, 2};
        jagged[2] = new int[]{1, 2, 3};
    }
}
```

4.6 Array of Objects (Arrays + OOP together)

```
class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    void display() {
```



```

        System.out.println(rollNo + " - " + name);
    }
}

public class ArrayOfObjects {
    public static void main(String[] args) {
        Student[] students = new Student[3];

        students[0] = new Student("Aarav", 1);
        students[1] = new Student("Priya", 2);
        students[2] = new Student("James", 3);

        for (Student s : students) {
            s.display();
        }
    }
}

```

4.7 Common Array Utilities (java.util.Arrays)

```

import java.util.Arrays;

public class ArrayUtils {
    public static void main(String[] args) {
        int[] nums = {5, 2, 9, 1, 7};

        Arrays.sort(nums); // sorts in place
        System.out.println(Arrays.toString(nums)); // [1, 2, 5, 7, 9]

        int index = Arrays.binarySearch(nums, 7); // search (array must be
sorted)
        System.out.println("Index of 7: " + index);

        int[] copy = Arrays.copyOf(nums, 3); // copy first 3 elements
        System.out.println(Arrays.toString(copy));

        int[] filled = new int[5];
        Arrays.fill(filled, 9); // fill all with 9
        System.out.println(Arrays.toString(filled));

        boolean equal = Arrays.equals(nums, copy); // element-wise
comparison
        System.out.println("Equal: " + equal);
    }
}

```

4.8 Key Properties of Arrays

- Fixed size once created (cannot grow/shrink — use `ArrayList` / `Vector` for dynamic size)
 - Zero-indexed
 - Stored in contiguous memory → $O(1)$ random access by index
 - `arrayName.length` gives size (no parentheses — it's a field, not a method)
 - Accessing an invalid index throws `ArrayIndexOutOfBoundsException`
 - Arrays are **objects** in Java (stored on heap), even for primitive types
-

5. Putting It Together — Encapsulation + Abstraction + Arrays

```
abstract class Employee {
    private String name;    // encapsulated field
    private double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    public String getName() { return name; }
    public double getBaseSalary() { return baseSalary; }

    // abstraction: subclasses define their own bonus logic
    abstract double calculateBonus();

    double totalPay() {
        return baseSalary + calculateBonus();
    }
}

class Manager extends Employee {
    Manager(String name, double baseSalary) {
        super(name, baseSalary);
    }

    @Override
    double calculateBonus() {
        return getBaseSalary() * 0.20;
    }
}
```

```

class Developer extends Employee {
    Developer(String name, double baseSalary) {
        super(name, baseSalary);
    }

    @Override
    double calculateBonus() {
        return getBaseSalary() * 0.10;
    }
}

public class Company {
    public static void main(String[] args) {
        // array of abstract type holding different concrete objects
        Employee[] employees = new Employee[2];
        employees[0] = new Manager("James", 90000);
        employees[1] = new Developer("Priya", 70000);

        for (Employee e : employees) {
            System.out.println(e.getName() + " total pay: " + e.totalPay());
        }
    }
}

```

This example shows all three concepts together:

- **Encapsulation:** `name` and `baseSalary` are `private`, accessed via getters
- **Abstraction:** `calculateBonus()` is abstract — each subclass hides its own bonus formula
- **Arrays:** `Employee[]` array stores different subclass objects, iterated polymorphically

6. Quick Revision Summary

- **Encapsulation** = data hiding via `private` fields + public getters/setters → protects state, enables validation
- **Abstraction** = hiding implementation, exposing only what's necessary → via `abstract class` (partial) or `interface` (full)
- **Arrays** = fixed-size, indexed, contiguous, same-type collections → `int[]`, multi-dimensional, array of objects, `java.util.Arrays` utilities